Linux System Administration (E1ITA307)

Linux Infrastructure Deployment Project Report

On

LINUX STORAGE MONITORING SYSTEM

(Black and White Cats)

Submitted by

SHASHANK PATEL (2411021061117)

PANKAJ PANDURANG TIKE (2411021061162)

ANMOL PODDAR (2411021061144)

SHIKHAR SRIVASTAVA (2411021061140)

In partial fulfillment of the award of the degree of

BACHELOR OF TECHNOLOGY

In

Information Technology

Under the Supervision of

Mrs. KUSUMA J.

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY

BENGALURU

Oct 2026



DEPARTMENT OF INFORMATION TECHNOLOGY

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the Project work entitled **"Linux Storage Monitoring System"** submitted by Shashank Patel [2411021061117], Pankaj Pandurang Tike [2411021061162], Anmol Poddar [2411021061144] and Shikhar Srivastava [2411021061140] in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology of Alliance University, is a bonafide work during the academic year 2026-2027.

Mrs. Kusuma J. Course Handling Faculty	Prof. Ananthanagu U Course Coordinator	Dr. Shekhar R Professor & HoD

Table of Contents

1. Introduction 4
1.1 Aim 4
1.2 Background 4
1.3 Problem Statement 4
1.4 Objectives 4
1.5 Team and Responsibilities 5
1.6 Scope of the Project 5
1.7 Organisation of the Report 5
2. Methodology 6
2.1 Tools and Technologies Used 6
2.2 Approach / Development Process 6
3. Linux System Administration – Design and Implementation 7
3.1 System Architecture 7
3.2 Linux Environment Setup 8
3.3 User and Group Management 10
3.4 File and Directory Management 11
3.5 Package and Process Management 12
3.6 Storage and File System Configuration 13
3.6.4 Logical Volume Management (LVM) 15
3.7 Backup and Automation 16
3.8 Security and Access Control 18
4. Results and Testing 19
4.1 Implementation Results 19
4.2 Testing and Validation 20
4.3 Observations 21
5. Conclusion and Future Scope 22
5.1 Conclusion 22
5.2 Future Scope 22
References 23
Appendices 24
Appendix A: Dashboard Pages and API 24
Appendix B: Troubleshooting 24
Appendix C: Scripts 25

1. Introduction

1.1 Aim

The main aim of this mini project is to design, develop and implement a **Linux Storage Monitoring System** (Black and White Cats) that automatically monitors disk space, inode usage and storage growth, predicts when a disk will become full, raises alerts when predefined thresholds are crossed, and presents everything in a daily report and a live web dashboard.

1.2 Background

Linux is widely used in servers, cloud infrastructure and development environments. Proper storage management is an important part of system administration, because a full disk or an exhausted inode table can stop applications from creating files, even when the rest of the system looks healthy. Commands such as `df` and `du` show only the present moment and only when someone runs them.

Our system uses a small Node.js service (no runtime dependencies) and the same `statfs` system call that `df` uses, so every number can be checked against a tool administrators already trust. It runs on Linux, macOS and Windows.

1.3 Problem Statement

Servers fail quietly when a disk or its inode table fills up. Administrators need a tool that **continuously watches disk space, inodes and growth, and warns them before the outage**, instead of discovering storage failures after they occur.

1.4 Objectives

- To monitor disk space and inode usage of every mounted file system automatically.
- To keep up to 30 days of storage history and estimate daily growth and days until full.
- To configure warning and critical thresholds for space, inodes and growth.
- To send clear, non-repetitive alerts through logs, syslog, webhooks or a custom command.
- To automate monitoring and generate daily storage reports.
- To show live information on a web dashboard and demonstrate basic backup and recovery.

1.5 Team and Responsibilities

Table 1.1: Team members and roles

Member	Register No.	Role	GitHub
Shashank Patel	2411021061117	Linux System Administrator	allenasat044-prog
Pankaj Pandurang Tike	2411021061162	Storage & Backup Administrator	pankajt25
Anmol Poddar	2411021061144	Automation & Security Engineer	ANMOL16-IT
Shikhar Srivastava	2411021061140	Testing, Documentation & Presentation Lead	FFloverSHIKHAR2006

1.6 Scope of the Project

The project **watches** storage; it does not create or repair it. Logical volumes, backups and file systems are set up by the administrator, and the monitor reports on their health. Chapters 3.6 and 3.7 therefore include a recommended LVM layout and a backup routine, to show how the monitor's warnings are put to use on a real server.

1.7 Organisation of the Report

Table 1.2: Report organisation

Chapter	Content
1. Introduction	Aim, background, problem statement, objectives and team
2. Methodology	Tools, technologies and the development process
3. Design and Implementation	Architecture, environment, users, files, packages, storage, backup and security
4. Results and Testing	Sample results, validation table and scope of testing
5. Conclusion and Future Scope	Summary of the work and possible improvements
References and Appendices	Sources, API, troubleshooting and scripts

2. Methodology

2.1 Tools and Technologies Used

2.1.1 Software Requirements

Table 2.1: Software used

Component	Used for
Linux (test VM)	Operating system used for development and testing
Node.js 18.15 or newer	Runs the monitoring service; no `npm install` is needed to run the prebuilt project
React, TypeScript, Tailwind CSS, Vite	Live web dashboard (shipped prebuilt in `dist/`)
Linux utilities	`df`, `du`, `mount`, `logger`, `rsync`, `systemctl`, `journalctl`, systemd, logrotate; Git for version control
systemd and logrotate	Start at boot, restart on failure, rotate the alert log

2.1.2 Hardware Requirements

Any laptop, desktop or virtual machine that runs a modern Linux distribution is enough. The monitor is very light: a few megabytes of memory and a little disk space for 30 days of history, reports and backups.

2.1.3 Other Tools and Resources

- **Configuration file:** `config/config.json` stores thresholds, interval and alert channels.
- **Terminal and logs:** commands and `journalctl` are used to run, inspect and verify the system.

2.2 Approach / Development Process

The project was developed in eight stages.

Step 1: **Problem identification:** disk full, inode exhaustion, silent growth and missing history.

Step 2: **Requirement analysis:** space, inodes, growth, alerts, reports, dashboard, low privilege.

Step 3: **Environment setup:** one project folder for server, configuration, data, reports and website.

Step 4: **System configuration:** thresholds, service user, systemd unit and log rotation.

Step 5: **Implementation:** four modules, HTTP API and dashboard.

Step 6: **Automation:** 30-second sampling, daily report, history pruning, cron backup.

Step 7: **Testing:** accuracy against `df`, growth and alert tests with `dd`, and a build check.

Step 8: **Documentation:** diagrams, configuration and results collected in this report.

3. Linux System Administration – Design and Implementation

3.1 System Architecture

The system is one Node.js process. The operating system is its only data source, plain files are its only storage, and the browser is its only client. Figure 3.1 shows the components and how they connect.

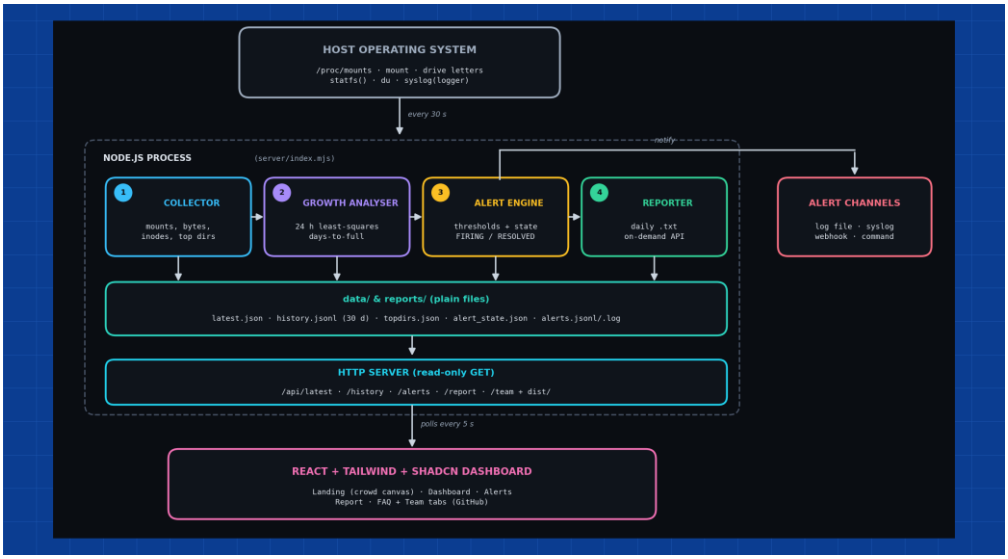


Figure 3.1: System architecture of Black and White Cats

Components and their roles

Table 3.1: Components and roles

Component	Role
1. Client / User	The administrator opens the dashboard, reads reports and reviews alerts.
2. Linux Server	Provides `/proc/mounts`, `statfs()`, `du`, syslog and systemd to the service.
3. Storage	Every mounted file system is monitored for space and inode usage.
4. Network	Data collection is local. The dashboard is served on `127.0.0.1:8080` by default.
5. Applications / Services	Collector, growth analyser, alert engine, reporter and HTTP server in one process.
6. Backup System	Nightly `rsync` backup to a separate `/backup` volume, protected by the monitor.
7. Monitoring and Automation	30-second sampling, 30-day history, daily report, state-based alerts.

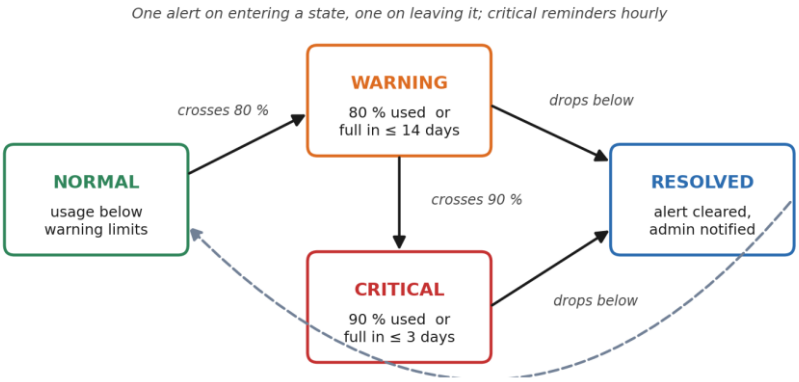


Figure 3.2: Alert states and transitions

Alerts follow this small state machine for every mount and problem kind, so the administrator is told once when a problem starts and once when it ends.

3.2 Linux Environment Setup

3.2.1 Platform and Requirements

The monitor runs on any Linux distribution that provides `/proc/mounts` and Node.js 18.15 or newer (needed for `fs.statfsSync`). Testing was done on a Linux virtual machine. Because it uses the same call as `df`, the results can be verified directly.

3.2.2 Installation Process

The project is unpacked into a folder and started with one command. The website is already built, so no download is needed.

```Bash
unzip black-and-white-cats.zip && cd bwc
node -v                  # needs 18.15 or newer
npm start                # same as: ./run.sh
# open http://127.0.0.1:8080 in the browser
```

```
$ cd ~/bwc && tree -L 2 -I 'node_modules|dist'
.
├── server
│   ├── monitor.mjs        # collector, growth, alerts, reports
│   └── index.mjs          # launcher: loop + HTTP API + website
├── scripts
│   ├── verify.mjs         # compare with df
│   └── dev.mjs
├── config
│   ├── config.json        # thresholds, interval, alert channels
│   └── team.json
├── src
│   ├── pages              # landing, dashboard, alerts, report, faq
│   └── components
├── data                   # history, state, alerts (created at run time)
├── reports                # daily text reports
├── dist                   # prebuilt website
└── run.sh  run.command  run.bat
```

Figure 3.3: Project directory structure

3.2.3 Network Configuration

Collecting storage data needs no network. The dashboard listens on `127.0.0.1` port 8080 by default; remote access is an explicit opt-in (`HOST=0.0.0.0`) and should be protected (section 3.8).

3.2.4 User Accounts

We recommend running the monitor as a dedicated, unprivileged user, because `statfs` works for any user and no root access is needed (section 3.3).

3.2.5 Required Packages

There are no runtime dependencies. Node.js is the only requirement. `rsync` and `logrotate` are optional and are used by the backup and log rotation examples.

3.2.6 System Services

A systemd unit makes the monitor start at boot and restart after a crash.

```
systemd unit
# /etc/systemd/system/storage-monitor.service
[Service]
User=storagemon
WorkingDirectory=/opt/bwc
Environment=PORT=8080 HOST=127.0.0.1
ExecStart=/usr/bin/node server/index.mjs --no-open
Restart=on-failure
NoNewPrivileges=true
ProtectSystem=strict
ReadWritePaths=/opt/bwc/data /opt/bwc/reports
[Install]
WantedBy=multi-user.target
```

```
Bash
sudo systemctl daemon-reload
sudo systemctl enable --now storage-monitor
systemctl status storage-monitor --no-pager
journalctl -t storage-monitor -n 20
```

3.3 User and Group Management

Note: this section describes the recommended account setup. These commands are an administration procedure and were not part of the automated tests.

3.3.1 User and Group Creation

A system user without a login shell runs the service, and a group lets administrators read reports and logs without root access.

```
Bash
sudo groupadd storageadm
sudo useradd -r -s /usr/sbin/nologin -G storageadm storagemon
id storagemon
getent group storageadm
```

3.3.2 File Ownership and Permissions

The project folder is owned by the service user and the admin group. The configuration file is set to `640` because it may hold a webhook URL or alert command, and the data and reports folders are set to `750`.

```Bash
sudo chown -R storagemon:storageadm /opt/bwc
sudo chmod 640 /opt/bwc/config/config.json
sudo chmod 750 /opt/bwc/data /opt/bwc/reports
ls -l /opt/bwc/config
```

3.3.3 Access Control

Members of `storageadm` can read the alert log and reports, other users have no access, and only the service user can write to `data/` and `reports/`. The systemd option `ProtectSystem=strict` enforces the same rule for the service.

3.3.4 Password Management

The monitor has no passwords of its own. Its service account has no login shell and no password, and administrators use their normal accounts and `sudo`.

3.4 File and Directory Management

3.4.1 Directory Structure

Table 3.2: Project folders

Folder	Purpose
`server/`	`monitor.mjs` (collector, growth, alerts, reports) and `index.mjs` (service and API)
`scripts/`	`verify.mjs` (accuracy check) and `dev.mjs` (development)
`config/`	`config.json` (thresholds, channels) and `team.json`
`data/`	Created at run time: history, state and alerts
`reports/`	Daily text reports
`dist/`	Prebuilt dashboard website

3.4.2 Files Written by the Monitor

Table 3.3: Data files

File	Content	Kept for
`data/latest.json`	Newest snapshot shown on the dashboard	Overwritten
`data/history.jsonl`	One usage sample per mount per cycle	30 days
`data/topdirs.json`	Largest directories, refreshed every 10 minutes	Overwritten
`data/alert_state.json`	Current state of every alert	Overwritten
`data/alerts.jsonl` / `.log`	Every FIRING and RESOLVED event	Rotated weekly
`reports/storage-report-DATE.txt`	Daily report for auditing	Kept

3.4.3 Searching Logs

Because all files are plain text, `grep`, `tail` and `find` are enough to investigate an alert. Figure 3.4 shows the real alert log entries from a run of the project.

```
$ tail -n 2 data/alerts.log
2026-10-10T16:05:14.052Z [FIRING] WARNING Shashanks-Laptop.local: /System/Volumes/Data
  projected full in 4.3 days at 69.61 GiB/day
2026-10-10T16:15:58.439Z [RESOLVED] WARNING Shashanks-Laptop.local: /System/Volumes/Data
   growth back to normal (was warning)
```

Figure 3.4: Alert log entries (sample run)

```
$ tail -n 1 data/history.jsonl
{"ts":1791649787.402,"m":{"/System/Volumes/iSCPreboot":[18231296,36,3.48],"/System/Volum
  es/Data":[172083281920,1082948,34.81]}}
```

Figure 3.5: One history record: timestamp, then [used bytes, used inodes, usage %] per mount

3.5 Package and Process Management

3.5.1 Package Management

The project has no runtime dependencies, so nothing has to be installed to run it. Development tools (React, Vite, Tailwind) are needed only to rebuild the website. The npm scripts below are the main commands.

Table 3.4: npm scripts

Command	Purpose
`npm start`	Collector, API and dashboard in one process
`npm run monitor`	One scan; exit code 0 / 1 / 2 (OK / warning / critical)
`npm run report`	Print and save the full text report
`npm run verify`	Cross-check every mount against `df`
`npm run build`	Type-check and rebuild the website into `dist/`
`npm run dev`	API server and Vite dev server together

3.5.2 Process Management

The monitor is a single process. Administrators can check it, restart it after a configuration change, or stop it with standard commands.

```bash
</>Bash
pgrep -af "server/index.mjs"          # is the monitor running?
sudo systemctl restart storage-monitor   # pick up a new config.json
sudo systemctl stop storage-monitor
```

3.5.3 Service Management and Logs

Under systemd, `systemctl status` shows whether the service is running, and `journalctl -t storage-monitor` shows the alerts it sent to syslog.

```
</>run.sh
#!/usr/bin/env bash
# macOS / Linux launcher. On macOS you can also double-click run.command
cd "$(dirname "$0")"
command -v node >/dev/null || { echo "Install Node.js >= 18.15"; exit 1; }
if [ ! -f dist/index.html ]; then npm install --no-audit --no-fund && npm run build ||
  exit 1; fi
exec node server/index.mjs "$@"
```

The launcher above (`run.sh`) checks that Node.js is installed, builds the website only if it is missing, and then starts the server.

3.6 Storage and File System Configuration

3.6.1 How the Monitor Reads Storage

For each mount, `statfs()` gives the block counts. The monitor calculates `use% = used / (used + avail)`, which is identical to `df`, and `inodes_used = files − ffree`. Pseudo file systems, `tmpfs` and container mounts are skipped, and bind mounts are collapsed by device id.

3.6.2 Threshold Configuration

Warning and critical limits are set in `config/config.json`.

```
$ cat config/config.json
{
  "interval_seconds": 30,
  "warn_pct": 80,
  "crit_pct": 90,
  "inode_warn_pct": 80,
  "inode_crit_pct": 90,
  "days_to_full_warn": 14,
  "days_to_full_crit": 3,
  "history_days": 30,
  "growth_window_hours": 24,
  "min_samples": 3,
  "min_span_seconds": 60,
  "du_every_seconds": 600,
  "include_tmpfs": false,
  "exclude_mount_prefixes": ["/proc", "/sys", "/dev", "/run", "/snap",
    "/var/lib/docker/", "/var/lib/kubelet/"],
  "alerts": {
    "reminder_minutes": 60,
    "log_file": true,
    "syslog": true,
    "webhook": { "enabled": false, "url": "" },
    "command": { "enabled": false, "run": "echo \"$ALERT_STATE $ALERT_LEVEL
      $ALERT_MESSAGE\" | mail -s \"Storage alert\" admin@example.com" }
  }
}
```

Figure 3.6: Monitoring configuration (config/config.json)

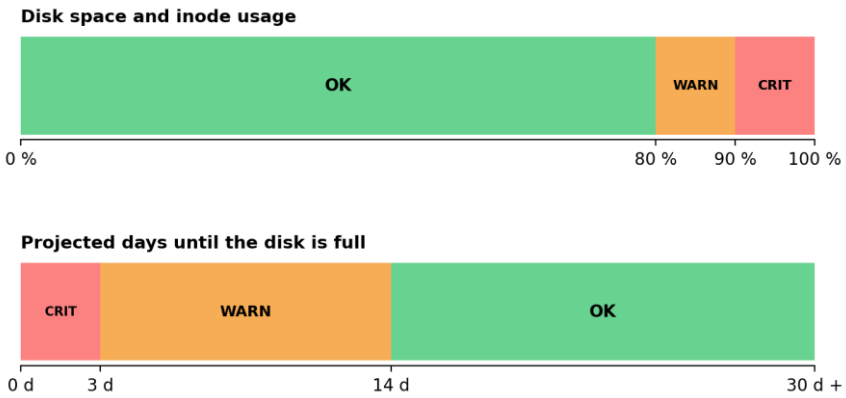


Figure 3.7: Alert limits for space, inodes and days to full

3.6.3 Growth Prediction

A least-squares line over the last 24 hours gives growth per day. Dividing the free space by it gives the days until the disk is full. In Figure 3.8 the disk is only about 70 % full, so a percentage check stays silent, yet the trend shows it will be full in about two days.

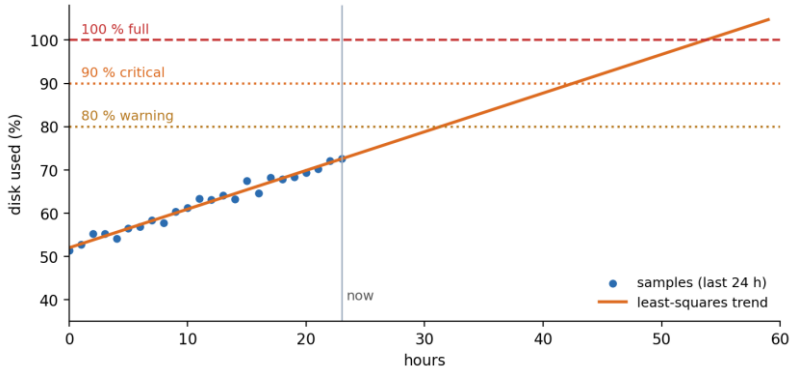


Figure 3.8: Growth prediction (illustrative example)

3.6.4 Logical Volume Management (LVM)

The monitor does not create volumes; it watches whatever is mounted, and a logical volume appears as an ordinary file system. LVM makes an alert actionable: when growth predicts a full disk, the administrator extends the volume **online** (Figure 3.9). **Note:** these commands are a reference lab procedure and are destructive on the disk they are pointed at; they were not run in the automated tests.

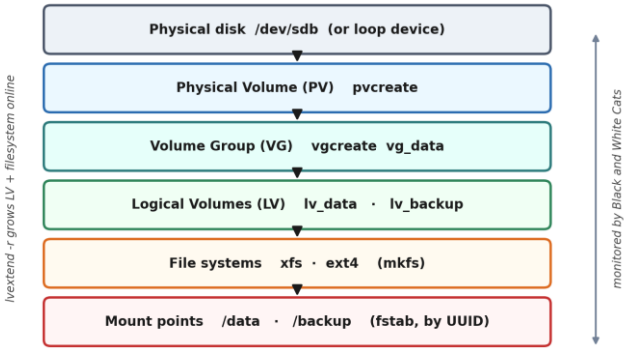


Figure 3.9: LVM layers from physical disk to mount point

```bash
# production layout on a spare disk /dev/sdb
sudo pvcreate /dev/sdb
sudo vgcreate vg_data /dev/sdb
sudo lvcreate -L 20G -n lv_data   vg_data
sudo lvcreate -L 10G -n lv_backup vg_data
sudo mkfs.xfs  /dev/vg_data/lv_data
sudo mkfs.ext4 /dev/vg_data/lv_backup
# mount by UUID in /etc/fstab, then:
sudo mount -a && df -hT /data /backup
```

```bash
# when the monitor warns, grow volume + file system online
sudo vgs && sudo lvs
sudo lvextend -r -L +5G /dev/vg_data/lv_data
# inode-heavy volume: one inode per 8 KiB
sudo mkfs.ext4 -i 8192 /dev/vg_data/lv_small_files
```

`xfs` can grow but never shrink, so `ext4` is safer where shrinking may be needed. The `ext4` inode count is fixed when the file system is created, which is why volumes for millions of small files are made with `-i 8192`.

3.7 Backup and Automation

3.7.1 Backup Strategy

The monitor watches capacity; it does not run backups. As the storage administrator's part, we use a nightly incremental `rsync` backup with hard links, so unchanged files cost almost no extra space. Fourteen days are kept, and the backup volume is mounted at `/backup`.

3.7.2 Backup Script and Schedule

```bash
#!/usr/bin/env bash   # nightly-backup.sh (root, chmod 750)
set -euo pipefail
DST=/backup/$(date +%F); mkdir -p "$DST"
rsync -a --delete --link-dest=/backup/latest /data/ "$DST"/
ln -sfn "$DST" /backup/latest
find /backup -maxdepth 1 -name '20*' -mtime +14 -exec rm -rf {} +
cd /opt/bwc && npm run --silent monitor || logger -t backup warn
```

```bash
# crontab -e   (root)
30 1 * * *    /usr/local/bin/nightly-backup.sh >> /var/log/backup.log
*/15 * * * *  cd /opt/bwc && npm run --silent monitor
```

3.7.3 How Backup and Monitor Work Together

The backup grows `/backup` every night. The monitor notices the growth, predicts the day the volume will fill, and warns the administrator in advance. After the volume is extended, the alert resolves by itself (Figure 3.10).

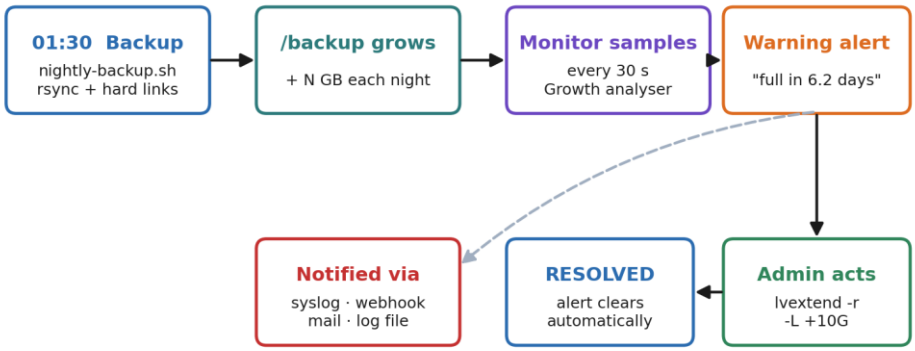


Figure 3.10: Backup and monitor working together

3.7.4 Recovery Procedure

Recovery means restoring selected files into a separate folder and checking that they match the originals. An empty `diff` result means there are no differences.

```bash
ls /backup                                  # list dated backups
mkdir -p /tmp/restore
rsync -a /backup/latest/ /tmp/restore/        # restore into a safe folder
diff -r /data /tmp/restore && echo "no differences"
```

3.7.5 Built-in Automation

Table 3.5: Automation shipped with the project

Mechanism	What it does
Collector loop	Samples automatically; logs a failed cycle and carries on
Daily report	The first sample of each day writes `reports/storage-report-DATE.txt`
Alert de-duplication	One FIRING and one RESOLVED message; critical reminders hourly
Exit codes	`npm run monitor` returns 0 / 1 / 2 for cron, Nagios or Task Scheduler

3.8 Security and Access Control

Table 3.6: Security controls

Control	Detail
Least privilege	Needs no root; capacity is read through `statfs`; runs as a dedicated user
Local by default	Binds to `127.0.0.1`; remote access is an explicit opt-in
Read-only API	GET only, so nothing in the UI can change the system
No shell injection	`du`, `logger` and `mount` are started with `execFile` and argument arrays
Path-traversal guard	Static files must resolve inside `dist/`
Hardened service	`NoNewPrivileges`, `ProtectSystem=strict`; atomic file writes

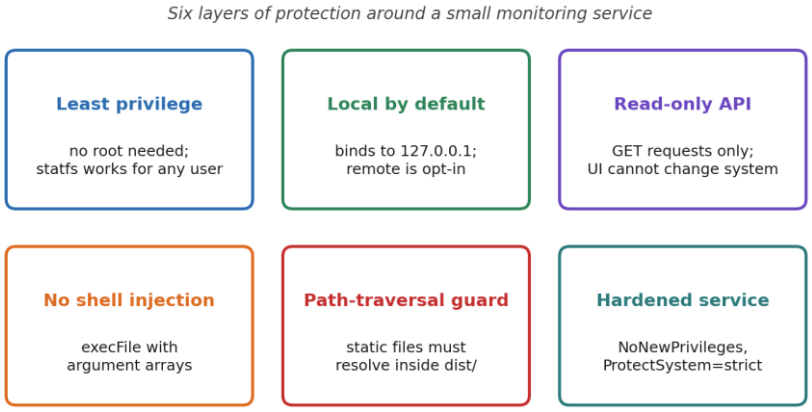


Figure 3.11: Security measures

3.8.1 Firewall and SSH

By default the dashboard listens only on localhost, so no firewall rule is needed. If remote access is required, port 8080 should be opened only to a trusted network. Administrators reach the server over SSH; the monitor itself does not use or need SSH.

```bash
sudo ufw status verbose
# only if remote access is really needed:
sudo ufw allow from 192.168.1.0/24 to any port 8080 proto tcp
systemctl status ssh --no-pager
```

3.8.2 Known Limits

There is no login. If the dashboard is exposed beyond localhost, it should sit behind a VPN or an authenticating reverse proxy, and only administrators should edit `config/config.json`.

4. Results and Testing

4.1 Implementation Results

The repository contains output from a real run of the monitor on a team member's laptop (macOS). Figure 4.1 shows the daily report from that run, and Figures 4.2 and 4.3 plot its data.

```
$ npm run report
================================================================================
 STORAGE REPORT  Shashanks-Laptop.local   10/10/2026, 9:57:47 pm
 darwin arm64  Darwin 25.6.0   uptime 3.34 days
================================================================================

MOUNT                TYPE           SIZE       USED       FREE   USE%  INODE%  STATUS
--------------------------------------------------------------------------------
/System/Volumes/iSCPr apfs     500.00 MiB  17.39 MiB 482.61 MiB    3.5     0.0  OK
/System/Volumes/Data  apfs     460.43 GiB 160.25 GiB 300.18 GiB   34.8     0.0  OK

GROWTH (least-squares over last 24h of samples)
--------------------------------------------------------------------------------
/System/Volumes/iSCPreboot 0 B/day, 0 inodes/day, not growing
/System/Volumes/Data   2.62 GiB/day, -13245 inodes/day, full in 114.6 days

LARGEST DIRECTORIES
--------------------------------------------------------------------------------
/System/Volumes/iSCPreboot:
      5.48 MiB  /System/Volumes/iSCPreboot/SFR
     44.00 KiB  /System/Volumes/iSCPreboot/SystemRecovery
     16.00 KiB  /System/Volumes/iSCPreboot/959AA465-741C-4F47-8A14-BEB4400FCBAE
      8.00 KiB  /System/Volumes/iSCPreboot/5F9234CF-C7F4-496F-B5F4-646CA2537157
      4.00 KiB  /System/Volumes/iSCPreboot/3D3287DE-280D-4619-AAAB-D97469CA9C71
/System/Volumes/Data:
```

Figure 4.1: Daily storage report (sample run)

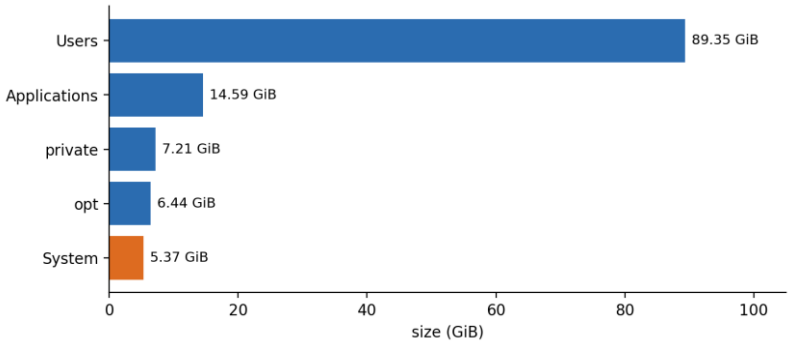


Figure 4.2: Largest directories found on the data volume (real sample)

In this run the monitor measured 2.62 GiB/day growth and 114.6 days until the 460 GiB volume would be full. Earlier, a burst of writing projected the disk full in 4.3 days; the monitor raised

a FIRING alert (Figure 3.4) and sent RESOLVED about ten minutes later when growth returned to normal.

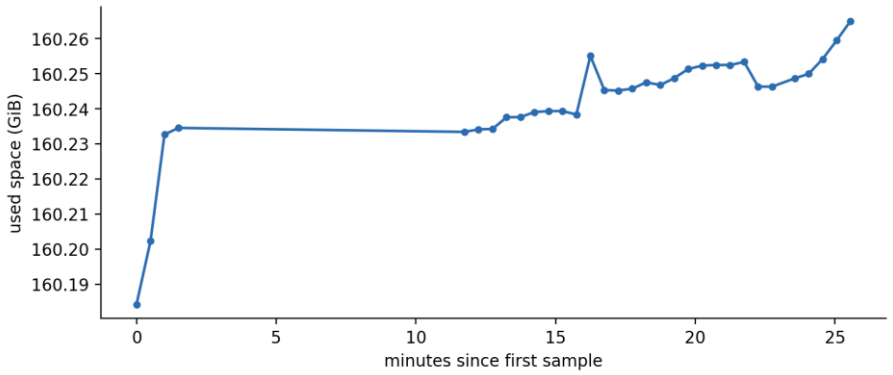


Figure 4.3: Used space on the data volume across 32 samples (real sample)

4.2 Testing and Validation

Table 4.1: Testing and validation

Test case	Expected result	Actual result	Status
Accuracy vs `df`	Used and available match within 64 MiB	Root matched to the byte; all mounts OK	Pass
Growth detection	Growth rate and days-to-full computed	Computed after writing 300 MB, then 700 MB with `dd`	Pass
Space alert	FIRING alert when limits are crossed	Limits lowered to 40 / 45 %: CRITICAL logged, exit code 2	Pass
Resolve flow	Alert clears when usage is normal	Thresholds restored: RESOLVED	Pass
Largest directories	Top directories listed	Matched `du`	Pass
Parsing	Linux and macOS mounts parsed	`/proc/mounts` and macOS `mount` samples correct	Pass
Build	No type errors	`tsc --noEmit` and `vite build` succeeded	Pass

4.2.1 Reproducing the Tests

```Bash
npm run verify                          # compare with df
dd if=/dev/zero of=/tmp/growth.bin bs=1M count=1000
npm run monitor; echo "exit=$?"         # 0 ok, 1 warning, 2 critical
```

4.2.2 Honest Scope of Testing

Automated testing was done on a Linux virtual machine. macOS discovery was verified against sample `mount` output, and Windows relies on drive-letter probing; all three use the same measurement code, but a first run there should be checked with `npm run verify` or `Get-PSDrive`. The LVM, backup and account commands in sections 3.3, 3.6 and 3.7 are reference procedures and were not part of these automated tests.

4.2.3 Testing Summary

The tests show that the monitor reports numbers that match `df`, detects fast growth before a disk is full, raises one alert per state change, and clears the alert automatically when the problem ends.

4.3 Observations

- The monitor's numbers matched `df`, so the figures on the dashboard can be trusted and cross-checked at any time.
- The growth analyser warned about a fast-growing disk long before a percentage check would have, which is the main benefit of the project.
- Alerts were not repeated every 30 seconds: one FIRING and one RESOLVED message were enough.
- The tool is lightweight, needs no database, and starts with a single command on Linux, macOS and Windows.
- Limitations: there is no login on the dashboard, and the LVM, backup and account steps are reference procedures that depend on the administrator's setup.

5. Conclusion and Future Scope

5.1 Conclusion

Black and White Cats turns raw `statfs` numbers into early, trustworthy warnings. By combining utilization, inode monitoring and growth prediction with state-based alerting, it guards against the two classic storage outages, the full disk and the exhausted inode table, with a small, dependency-free, cross-platform tool that an administrator can start with one command and verify against `df`.

The project also gave us practical experience in Linux system administration: services and systemd, users and permissions, logging and rotation, LVM and storage planning, backup and recovery, and basic security. We learned that a warning is only useful if it arrives early and arrives once, and that a monitoring tool should be read-only, local and unprivileged by default.

5.2 Future Scope

- **Notifications:** built-in email, Slack and Telegram senders instead of a custom command.
- **Security:** login and roles, so the dashboard can be used safely from other machines.
- **LVM-aware monitoring:** show volume groups and free extents, and suggest how much to extend.
- **Backup status:** show the last backup run, size and age on the dashboard.
- **Hardware health:** SMART and RAID status, and per-user quotas.
- **Multi-host monitoring:** agents on several servers with one central dashboard.
- **Integration and forecasting:** a Prometheus exporter, a Docker image and better forecasting than a straight line.

References

1. Black and White Cats – project source code and README: https://github.com/allenasat044-prog/LinuxCats
2. Linux manual pages: statfs(2), df(1), du(1), rsync(1), logger(1), logrotate(8), fstab(5), mkfs.ext4(8). https://man7.org/linux/man-pages/
3. LVM2 documentation: pvcreate(8), vgcreate(8), lvcreate(8), lvextend(8).
4. systemd documentation: systemd.service(5), systemd.exec(5).
5. Node.js documentation. https://nodejs.org/docs
6. React, Tailwind CSS and Vite documentation. https://react.dev, https://tailwindcss.com, https://vitejs.dev
7. Course material, Linux System Administration (E1ITA307), Alliance School of Advanced Computing, Alliance University.

Appendices

Appendix A: Dashboard Pages and API

The dashboard is polled every 5 seconds and has five pages. The read-only API behind it is listed in Table A.1.

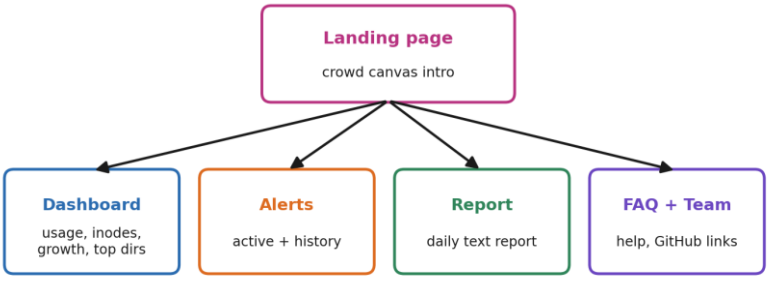


Figure A1: Pages of the web dashboard

Table A.1: HTTP API (read-only)

Endpoint	Returns
`GET /api/latest`	Latest snapshot: file systems, growth, alerts, summary
`GET /api/history?hours=24`	Usage samples (up to 300 points)
`GET /api/alerts?hours=168`	Active alerts and event history
`GET /api/report`	Current text report
`GET /api/team`	Team members

```Bash
curl -s http://127.0.0.1:8080/api/latest
curl -s "http://127.0.0.1:8080/api/history?hours=24"
curl -s "http://127.0.0.1:8080/api/alerts?hours=168"
curl -s http://127.0.0.1:8080/api/report
```

Appendix B: Troubleshooting

Table B.1: Common problems

Symptom	Fix
`Missing script: "start"`	You are in the wrong folder. `cd` into the folder that contains `package.json`.
`Port 8080 is busy`	Use another port: `PORT=8090 npm start` (Linux / macOS).
Landing page has no walking crowd	The sprite loads from a CDN, so connect to the internet.
Team tab shows no stats	The GitHub API is unreachable or rate-limited; the links still work.
`npm start` fails on Windows PowerShell	Use `npm.cmd start`, or double-click `run.bat`.

Appendix C: Scripts

The following scripts and units are part of the project or of its recommended deployment. The launcher below is `run.sh` and the development script is `scripts/dev.mjs`; the backup script, cron entries and systemd unit are shown in sections 3.2 and 3.7.

run.sh

```
#!/usr/bin/env bash
# macOS / Linux launcher. On macOS you can also double-click run.command
cd "$(dirname "$0")"
command -v node >/dev/null || { echo "Install Node.js >= 18.15"; exit 1; }
if [ ! -f dist/index.html ]; then npm install --no-audit --no-fund && npm run build ||
  exit 1; fi
exec node server/index.mjs "$@"
```

dev.mjs

```
// scripts/dev.mjs - npm run dev: API server + Vite dev server together
import { spawn } from "node:child_process";
const run = (a) => spawn(process.execPath, a, { stdio: "inherit" });
const api = run(["server/index.mjs", "--no-open"]),
      vite = run(["node_modules/vite/bin/vite.js"]);
const stop = () => { api.kill(); vite.kill(); process.exit(); };
process.on("SIGINT", stop); process.on("SIGTERM", stop);
```